

📱 5% > 100%: Breaking Performance Shackles of Full Fine-Tuning on Visual Recognition Tasks

Dongshuo Yin^{1,2*,‡}, Leiya Hu^{3*}, Bin Li⁴, Youqun Zhang⁴, Xue Yang^{1†}

¹Department of Automation, Shanghai Jiao Tong University,

²BNRist, Department of Computer Science and Technology, Tsinghua University,

³University of Chinese Academy of Sciences, ⁴Alibaba Group

yinds@mail.tsinghua.edu.cn, huleiya21@mails.ucas.ac.cn

{zhuyi.lb, youqun.zhangyq}@alibaba-inc.com, yangxue-2019-sjtu@sjtu.edu.cn

Abstract

Pre-training & fine-tuning can enhance the transferring efficiency and performance in visual tasks. Recent delta-tuning methods provide more options for visual classification tasks. Despite their success, existing visual delta-tuning art fails to exceed the upper limit of full fine-tuning on challenging tasks. To find a competitive alternative to full fine-tuning, we propose the **Multi-cognitive Visual Adapter (Mona)** tuning, a novel adapter-based tuning method. First, we introduce multiple vision-friendly filters into the adapter to enhance its ability for processing visual signals, while previous methods mainly rely on language-friendly linear filters. Second, we add the scaled layer-norm in the adapter to regulate the distribution of input features for visual filters. To fully demonstrate the practicality and generality of Mona, we conduct experiments on representative visual tasks, including instance segmentation on COCO, semantic segmentation on ADE20K, object detection on Pascal VOC, oriented object detection on DOTA/STAR, and image classification on three common datasets. Exciting results illustrate that Mona surpasses full fine-tuning on all these tasks by tuning less than 5% params of the backbone, and is the only delta-tuning method outperforming full fine-tuning on all tasks. For example, Mona achieves 1% performance gain on the COCO compared to full fine-tuning. Comprehensive results suggest that Mona-tuning is more suitable for retaining and utilizing the capabilities of pre-trained models than full fine-tuning. The code is publicly available on <https://github.com/Leiyi-Hu/mona>.

*Equal contribution.

†Corresponding author.

‡The work was initiated when the author was interning at Shanghai Jiao Tong University and completed when the author worked at Tsinghua University.

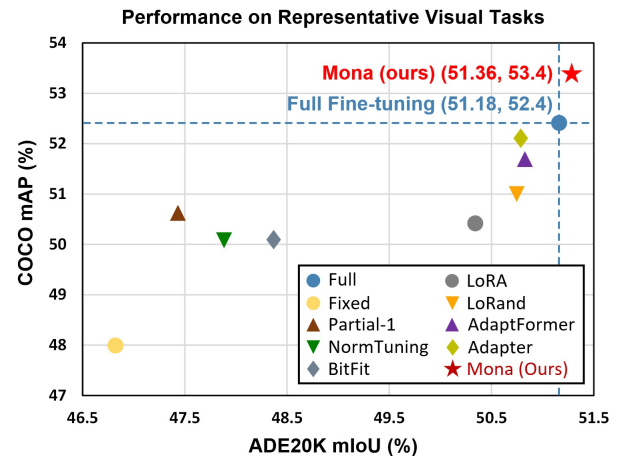


Figure 1. Comparisons of our method with full fine-tuning and recent delta-tuning art on representative visual tasks. Blue dashed line is the performance of full fine-tuning on ADE20K and COCO. The proposed Mona outperforms full fine-tuning on representative visual tasks, which promotes the upper limit of previous delta-tuning art. The results demonstrate that the adapter-tuning paradigm can replace full fine-tuning and achieve better performance in common visual tasks. Full fine-tuning is not the only preferred solution for transfer learning in the future.

1. Introduction

Pre-training & fine-tuning paradigm [1] can perform impressive transfer learning between homo-modal tasks, as has been demonstrated in computer vision (CV) [2, 3] and natural language processing (NLP) [4–6]. Pre-trained models are often trained by well-resourced and experienced teams with large amounts of clean data [7]. Exceptional pre-trained models can help hardware- and data-limited teams save plenty of training costs and train well-performing deep models on new tasks [8–11]. In the era of large models, the efficiency of tuning pre-trained models is an im-

portant issue. Full fine-tuning has been widely used with great success in CV tasks, which tunes all parameters in the pre-trained backbone as well as additional task-specific heads/necks during the training process. Many impressive CV art push the limit of visual tasks through pretraining & full fine-tuning. However, is full fine-tuning still the best way to fine-tune visual tasks now?

Apart from full fine-tuning, Delta tuning [12, 13] has recently attracted attention in NLP and CV tasks. Delta tuning comes from NLP, which tunes only part of the backbone network or extra lightweight structures for efficient transfer learning [12]. Delta tuning methods generally fix most backbone parameters and achieve comparable or even better performance than full fine-tuning on simple tasks (including classification tasks in NLP [14, 15] and CV [16–19]). VPT [16] first explores the potential of prompt-tuning on visual classification tasks. LoRand [20] pioneers adapter-tuning on dense predictions and reduces the gap between delta tuning and full fine-tuning on visual tasks. However, existing methods cannot outperform full fine-tuning on visual recognition tasks, including semantic and instance segmentation.

To challenge the dominance of full fine-tuning in CV, we propose Mona-tuning, a novel tuning paradigm based on **Multi-cognitive visual adapters (Mona)**. We analyse recent art and summarise two issues in existing visual adapters. First, the designs of existing CV adapters [17–19] follow linear adapters in NLP [21]. In fact, visual tasks process visual signals, which are significantly different from linguistic signals and have unique 2D convolutional operations [22–24]. Our experiments show that convolution-based filters can transfer visual knowledge from pre-trained models to other tasks better, so we propose a practical convolution-based adapter for visual tasks. Second, most existing adapters compress the upstream features with a single linear layer [17, 18]. Previous works claim that models have different cognition of features at different filter scales [25–27]. Thus, we employ multiple convolutional filters behind the adapter’s reduction layer to enhance the cognitive abilities of the adapters. We demonstrate the generality and superiority of Mona-tuning on plenty of representative visual tasks, including image classification, object detection, semantic segmentation, instance segmentation, and oriented object detection [28, 29]. We employ the SwinTransformer [2] series trained on ImageNet-22k [30] as pre-trained models. Extensive experiments indicate that the proposed method outperforms the traditional full fine-tuning paradigm both on simple image classification tasks and complex visual tasks. For example, Mona-tuning outperforms full fine-tuning on the COCO dataset [31] by 1% mAP. The results suggest that full fine-tuning is not the optimal choice for many visual tasks. As far as we known, Mona is the only Adapter-based tuning method that surpasses full fine-tuning on semantic segmentation, instance

segmentation, and oriented object detection. Figure 1 illustrates the superiority of the proposed method on the challenging instance segmentation and semantic segmentation tasks. Our contributions can be three-fold:

- We demonstrate that the adapter-based tuning can surpass full fine-tuning on visual tasks, and perform better than full fine-tuning with fewer new parameters.
- We propose Mona-tuning, a novel and practical training paradigm based on multi-cognitive visual adapters (Mona). Mona employs vision-friendly filters to optimise traditional linear adapters and improve the transferring efficiency of visual pre-trained knowledge through multiple cognitive perspectives.
- Extensive experiments demonstrate that Mona-tuning outperforms full fine-tuning and other recent art on representative visual tasks, including image classification, object detection, semantic segmentation, instance segmentation, and oriented object detection.

2. Related Work

2.1. Delta-tuning

The development of large models has produced dramatic shocks throughout artificial intelligence [32–34]. The efficiency of transfer learning attracts researchers’ interest [17–19, 35]. Delta tuning [12, 13, 21, 36, 37] (or parameter efficient fine-tuning, PEFT) is dedicated to improving the efficiency of fine-tuning. Delta-tuning methods can be divided into three groups [12, 38]. The first group fixes most of the parameters in the pre-trained backbone and fine-tune a small number of them, e.g., BitFit [39] tunes bias, Norm Tuning [40] tunes norm layers, and Partial-1 [41] only tunes the last block. The second group reparameterises some parameters in the pre-trained model, e.g. the LoRA [36] optimises low-rank subspaces. The third group fixes the pre-trained backbone’s original parameters and adds additional trainable structures, including prompt series [16, 42, 43] and adapter series [19, 44, 45]. Our experiments compare Mona with these three groups.

2.2. Computer Vision Meets Delta-tuning

Although derived from NLP, delta tuning is also explored in CV. VPT [16] is the first to introduce delta-tuning (prompt-tuning) to visual classification tasks. [46] adds adapters to a trainable backbone to improve performance rather than parameter efficiency. [47, 48] differentiates the parameters and fine-tunes some of the parameters in the extra structures. SSF [49] scales and shifts features for fine-tuning. [50] designs adapters that can be adjusted to a specific task. AdaptFormer [19] designs a parallel adapter structure to improve delta-tuning performance on visual classification. KAdaptation [45] optimises the adapter through the Kronecker product. The above art is the pioneer in visual

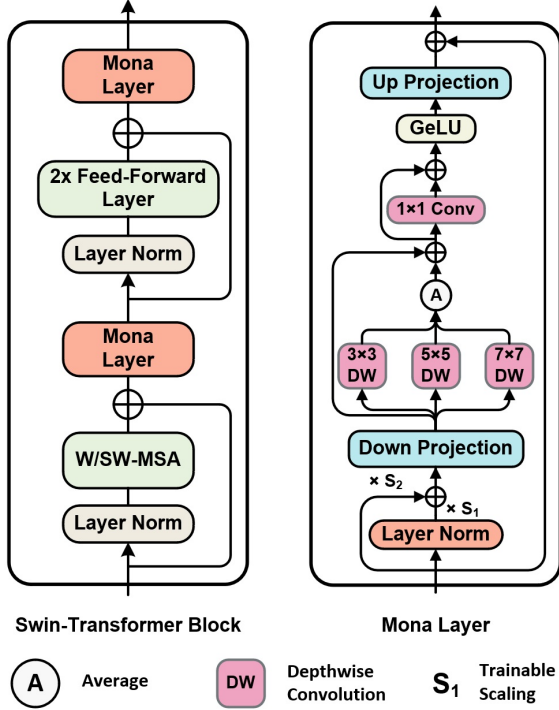


Figure 2. **Left:** The proposed Mona-tuning. We add Mona after MSA and MLP in each SwinBlock. The proposed method fixes the parameters of pre-trained layers and updates the parameters of Mona. **Right:** Details of Mona. Mona has a scaled LayerNorm before the down projection. A multi-cognitive convolutional filter group and an aggregation filter are behind the down projection. We add skip-connections at four places inside Mona to strengthen its adaptation capabilities. Mona enables the adapter-based fine-tuning paradigm to outperform full fine-tuning in typical visual tasks comprehensively.

tasks, revealing the potential of delta-tuning on visual classification. LoRand [20] brings impressive performance on dense prediction tasks via multi-branch low-rank adapters but still cannot surpass full fine-tuning on all visual recognition tasks. Recent art indicates that delta-tuning cannot completely replace full fine-tuning on vision tasks. Therefore, we propose Mona-tuning, an alternative to full fine-tuning for more visual tasks, which outperforms full fine-tuning in both new parameter size and performance.

3. Methods

In this section, we present the proposed method in four parts, including the adapter-tuning, Mona, and parameter analysis.

3.1. Adapter-tuning

Previous work [20] discusses adapter fine-tuning, and we briefly introduce related concepts here. Full fine-tuning updates all parameters in the pre-trained backbone, while

adapter-tuning fixes the pre-trained parameters and updates the parameters in adapters. For dataset $D = \{(x_i, y_i)\}_{i=1}^N$, the optimization process of full fine-tuning and adapter-tuning can be expressed as Eq. 1 and Eq. 2:

$$\theta \leftarrow \arg \min_{\theta} \text{loss}(D, \theta), \quad (1)$$

$$\omega \leftarrow \arg \min_{\omega} \text{loss}(D, \theta_F, \omega), \quad (2)$$

where loss is the training loss, θ represents parameters of the whole framework, and θ_F is the fixed parameters in adapter-tuning. ω represents updated parameters in adapter-tuning, including parameters in adapters and outside the backbone.

3.2. Mona

Typical linear adapters suffer from two problems when applied to visual tasks. First, fixed layer parameters cannot be fine-tuned to match the data distribution of new tasks, resulting in a biased feature distribution passed to the adapter. Therefore, it is important for the adapter to optimize its input distribution from fixed layers. Second, Vanilla adapter [21] is designed for natural language signals and is not optimized for visual signals. Previous CV adapter art [16–19] is based on linear filters (mainly including down projection, nonlinear activation, up projection, and skip connections), which is not efficient for transferring visual knowledge. To address these two issues, we perform input optimization and design multi-cognitive visual filters.

Input Optimization. We enable Mona to adjust the input distributions and the proportion of inputs from the fixed layers. Specifically, we add a norm layer and two learnable weights, s_1 and s_2 , to the top end of Mona to adjust the input distribution. Previous work indicates that normalization [51] helps to stabilize the forward input distribution and the backpropagated gradient. We find in practice that LayerNorm (LN) [51] is better than BatchNorm [52], so we employ LN in Mona. Figure 2 illustrates our design, which can be formulated as:

$$x_{\text{norm}} = s_1 \cdot |x_0|_{\text{LN}} + s_2 \cdot x_0, \quad (3)$$

where $|\cdot|_{\text{LN}}$ denotes LayerNorm and x_0 denotes the original input of Mona.

Multi-Cognitive Visual Filters. For visual cognition, human eyes process visual signals from different scales and integrate them for better understanding [53–55]. Adapters should also process upstream features from multiple cognitive perspectives for better performance on downstream tasks. We introduce multiple convolutional filters to Mona to increase the cognitive dimension. Instead of standard convolutions, Depth-Wise Convolutions [55] (DWConv) are employed in Mona to minimize additional parameter sizes. Specifically, the upstream features go through three

DWConv filters after down projection. Convolution kernels are 3×3 , 5×5 and 7×7 . We compute the average results from three filters and aggregate features with a 1×1 convolution. Skip-connections are added to two types of convolutions. We use three depth-wise convolutions with weight $\omega_{dw}^i \in \mathbb{R}^{C_{in}^D \times K_i \times K_i \times C_{out}^D}$ ($i \in 1, 2, 3$) for the first multi-filter convolution and a point-wise convolution with weight $\omega_{pw}^i \in \mathbb{R}^{C_{in}^P \times 1 \times 1 \times C_{out}^P}$ for the second convolution. The above two convolution steps can be formulated as follows:

$$\begin{aligned} f_{dw} &= x + avg(\sum_{i=1}^3 \omega_{dw}^i \hat{\otimes} x), \\ f_{pw} &= x + \omega_{pw} \otimes x, \end{aligned} \quad (4)$$

where $\hat{\otimes}$ and $\otimes$ denote depth-wise and point-wise convolution. Then, features are nonlinearized by GeLU and recovered by up projection. The overall calculation process of Mona can be formulated as follows:

$$x = x_0 + U^l \sigma(f_{pw}(f_{dw}(D^l(x_{norm}))), \quad (5)$$

where D^l and U^l denote down and up projections of the l^{th} adapter, and σ denotes GeLU activation.

3.3. Parameter Analysis

The parameters of Mona come from LN, scaling factors, linear layers, DWconv, and 1×1 conv. Assuming that the input dimension of the adapter is m and the dimension after down projection is n , the parameters of the LN and scaling factors are $2m + 2$, the parameters of the two linear layers are $2mn + m + n$, the parameters of the DWConv layer are $(3^2 + 5^2 + 7^2)n = 83n$, and the PWConv is n^2 . The total parameter of each Mona module are:

$$(2n + 3)m + n^2 + 84n + 2. \quad (6)$$

For each block, all Mona parameters are: $2 \times ((2n + 3)m + n^2 + 84n + 2)$. We set the value of n to a constant (64) to reduce parameters in Mona.

4. Experiments

We implement sufficient experiments on multiple representative visual tasks to demonstrate the superiority of Mona-tuning. This section includes experimental settings, results, convergence analysis, and some ablation experiments. Hyperparameters and detailed settings for training are presented in the Supplementary Material.

4.1. Datasets

Object Detection. Pascal VOC 0712 [56] has 16k/5k training/validation images and is used for object detection tasks. We employ Swin-Large + RetinaNet for training. The evaluation metric for object detection task is the most commonly used AP_{box} .

Semantic Segmentation. ADE20K [57] is the most widely used semantic segmentation dataset containing 20K training and 2K validation images. We employ Swin-Large + UpNet for experiments on semantic segmentation. The evaluation metric is the most commonly used mIoU.

Instance Segmentation. MS COCO [31] is a representative instance segmentation dataset with 118k training images and 5k validation images. We employ Swin-Base + Cascade Mask RCNN for training. Evaluation metrics for instance segmentation task are AP_{box} and AP_{Mask} .

Oriented Object Detection. Oriented object detection considers angle information in the annotation and inference process, which can effectively improve the performance and efficiency of object detection in fields like remote sensing. This task requires more accurate annotation information and more complex detection models, which is more challenging than horizontal object detection. Two representative remote sensing datasets, DOTA [58] and STAR [59], are selected for our experiments. We also experiment with multiple detection frameworks on the more challenging STAR dataset. The metric here is AP_{box} .

Image Classification. Classification tasks have been well studied in previous art. We also conduct experiments on Oxford 102 Flower [60], Oxford-IIIT Pet [61], and VOC 2007 Classification dataset [62] to increase the broadness of our experiments. The top-1, top-5, and average accuracy of each method are reported.

4.2. Pre-trained Models and Toolkits

The Swin Transformer series [2] are employed as the backbone for all experiments. The pre-trained models are trained on ImageNet-22k [30], and toolkits like MMDetection [63], MMSegmentation [64], MMRotate [65] and MMClassification [66] are used for verification. The image resolution of the pre-trained task is 224×224 . Most tasks employ Swin-Large as the backbone. Backbones for COCO, DOTA, and STAR are Swin-Base, considering the memory consumption of these tasks.

4.3. Baselines

We compare Mona with multiple recent methods. Baselines can be grouped into methods without or with extra structure:

- Without extra structure:
 - FULL: Update all parameters in the framework.
 - FIXED: Fix the backbone and update other parameters.
 - BITFIT [39]: Update bias in backbone and parameters outside of backbone.
 - NORMTUNING [40]: Update norm layers in backbone and parameters outside the backbone.
 - PARTIAL-1 [41]: Update the last block in the backbone and parameters outside the backbone.
- With extra structure:

Swin-B (89M)	Trained* Params	%	Δ_{Full}	Extra Structure	COCO (Cascade Mask R-CNN)			
					AP _{Box}	Δ_{Full}	AP _{Mask}	Δ_{Full}
Baselines								
FULL	89.14 M	100.00 %	-	×	52.40 %	-	45.10 %	-
FIXED	0.00 M	0.00 %	- 100.00 %	×	48.00 %	- 4.40 %	41.60 %	- 3.50 %
BITFIT	0.21 M	0.23 %	- 99.77 %	×	50.10 %	- 2.30 %	43.60 %	- 1.50 %
NORMTUNING	0.06 M	0.07 %	- 99.93 %	×	50.10 %	- 2.30 %	43.50 %	- 1.60 %
PARTIAL-1	12.95 M	14.53 %	- 85.47 %	×	50.60 %	- 1.80 %	43.70 %	- 1.40 %
ADAPTER	3.19 M	3.58 %	- 96.42 %	✓	52.10 %	- 0.30 %	45.00 %	- 0.10 %
LoRA	3.06 M	3.43 %	- 96.57 %	✓	50.40 %	- 2.00 %	43.90 %	- 1.20 %
ADAPTFORMER	1.60 M	1.79 %	- 98.21 %	✓	51.70 %	- 0.70 %	44.60 %	- 0.50 %
LoRAND	4.68 M	5.23 %	- 94.77 %	✓	51.90 %	- 0.50 %	44.70 %	- 0.40 %
Our Method								
MONA	4.16 M	4.67 %	- 95.33 %	✓	53.40 %	+ 1.00 %	46.00 %	+ 0.90 %

Table 1. **Results of baselines and our methods on COCO benchmarks.** Swin-B is employed as the pre-trained model here. We present the numbers and percentages of trainable backbone parameters on the left and all the performances on the right. * denotes the trainable parameters in backbones. The best AP in each column is bolded.

Swin-L (198M)	Trained* Params	%	Δ_{Full}	Extra Structure	Pascal VOC (RetinaNet)		ADE20K (UperNet)	
					AP _{Box}	Δ_{Full}	mIoU	Δ_{Full}
Baselines								
FULL	198.58 M	100.00 %	-	×	83.70 %	-	51.18 %	-
FIXED	0.00 M	0.00 %	- 100.00 %	×	83.80 %	+ 0.10 %	46.84 %	- 4.34 %
BITFIT	0.30 M	0.15 %	- 99.85 %	×	85.40 %	+ 1.70 %	48.37 %	- 2.81 %
NORMTUNING	0.10 M	0.05 %	- 99.95 %	×	85.50 %	+ 1.80 %	47.89 %	- 3.29 %
PARTIAL-1	28.77 M	14.53 %	- 85.47 %	×	85.50 %	+ 1.80 %	47.44 %	- 3.74 %
ADAPTER	4.61 M	2.33 %	- 97.67 %	✓	86.70 %	+ 3.00 %	50.78 %	- 0.40 %
LoRA	4.57 M	2.31 %	- 97.69 %	✓	85.40 %	+ 1.70 %	50.34 %	- 0.84 %
ADAPTFORMER	2.34 M	1.18 %	- 98.82 %	✓	86.60 %	+ 2.90 %	50.83 %	- 0.35 %
LoRAND	5.20 M	2.62 %	- 97.38 %	✓	86.90 %	+ 3.20 %	50.93 %	- 0.25 %
Our Method								
MONA	5.08 M	2.56 %	- 97.44 %	✓	87.30 %	+ 3.60 %	51.36 %	+ 0.18 %

Table 2. **Results of baselines and our methods on Pascal VOC and ADE20K benchmarks.** Swin-L is employed as the pre-trained model here. We present the numbers and percentages of trainable backbone parameters on the left and all the performances on the right. * denotes the trainable parameters in backbones. The best AP/mIoU in each column is bolded.

(The pre-trained layers in these baselines are fixed, and the adapter intermediate dimensions are all 64, following the AdaptFormer [19]):

- ADAPTER [21]: Add standard adapter layers after the MSA/MLP layers of each SwinBlock.
- LoRA [36]: Add parallel learnable matrices to multi-head attention weights.
- ADAPTFORMER [19]: Add parallel adapter layers with scale weights to each MLP layer.
- LoRAND [20]: Add LoRand++ ($\alpha=4$, $\beta=16$) layers after the MSA/MLP of each SwinBlock. LoRand++ has the best performance among its variants, so the most chal-

lenging setting is chosen for comparison.

4.4. Main Results

Instance segmentation on COCO is challenging. From Table 1, we find that Mona outperforms all PEFT baselines and is the only method that outperforms full fine-tuning even by 1%. COCO experiments effectively demonstrate the capability of the proposed method and show a better option than full fine-tuning in terms of storage and performance. Among delta-tuning methods, most baselines without extra structure can save more new parameters (except Partial-1), but their average performance is lower

Method	Flowers102		OxfordPets		VOC2007		Average	
	top-1 acc.	top-5 acc.	top-1 acc.	top-5 acc.	top-1 acc.	top-5 acc.	top-1 acc.	top-5 acc.
Baselines								
FULL	99.5772	99.8536	94.6579	99.6257	84.1276	96.9507	92.7876	98.8100
FIXED	99.3007	99.8374	94.2219	99.9182	85.0162	98.9499	92.8463	99.5685
BITFIT	99.5772	99.8211	95.3393	99.9182	85.6018	99.3336	93.5061	99.6910
NORMTUNING	99.5284	99.8374	95.2303	99.8910	85.5210	99.2528	93.4266	99.6604
PARTIAL-1	99.6585	99.8374	95.3938	99.8637	84.9354	98.6066	93.3292	99.4359
ADAPTER	99.5934	99.8536	95.3393	99.8092	87.0355	99.1317	93.9894	99.6144
LoRA	99.5446	99.8536	95.1485	99.8910	85.7028	99.3134	93.4653	99.6860
ADAPTFORMER	99.5609	99.8536	95.2576	99.8365	86.2884	99.2730	93.7023	99.6544
LoRAND	99.5725	99.8536	95.3515	99.8910	86.6534	99.3741	93.8591	99.7062
Our Method								
MONA	99.6764	99.9024	95.4765	99.9182	86.9709	99.5057	94.0413	99.7592

Table 3. Results of baselines and our methods on three classification datasets. Swin-L is employed as the pre-trained model here. We present top-1 accuracy (%) and top-5 accuracy (%) of each dataset. The best result in each column is bolded.

Swin-B (89M)	Oriented R-CNN (Faster R-CNN)		KLD (RetinaNet)	H2RBox-v2 (FCOS)
	DOTA-v1.0	STAR	STAR	STAR
Baselines				
FULL	78.31 %	38.63 %	30.33 %	30.29 %
FIXED	74.10 %	30.83 %	23.81 %	26.01 %
BITFIT	76.05 %	34.51 %	28.17 %	29.41 %
NORMTUNING	75.82 %	33.13 %	27.12 %	27.79 %
PARTIAL-1	75.72 %	33.96 %	28.53 %	28.89 %
ADAPTER	78.27 %	37.97 %	30.35 %	30.24 %
LoRA	75.91 %	33.80 %	27.48 %	28.95 %
ADAPTFORMER	77.43 %	35.95 %	29.36 %	30.11 %
LoRAND	77.65 %	36.44 %	29.83 %	28.85 %
Our Method				
MONA	78.44 %	39.45 %	30.90 %	31.34 %

Table 4. Results on DOTA and STAR benchmarks. Swin-B is employed as the pre-trained model here. The best AP in each column is bolded.

than that with extra structure. For baselines with additional structure, adapter-based approaches is superior to the reparameterization-based LoRA. Table 1 shows that LoRA performs well on NLP tasks but poorly on computer vision tasks. Table 1 indicates that the performance of delta-tuning is not directly related to parameter sizes. Partial-1 has the most trainable parameters, but its performance is significantly lower than that of adapter-based baselines. This result suggests that superior module design can effectively enhance the transferring efficiency of pre-trained models and reduce massive new parameters.

Table 2 shows the results of Pascal VOC (object detection) and ADE20K (semantic segmentation). Also, Mona

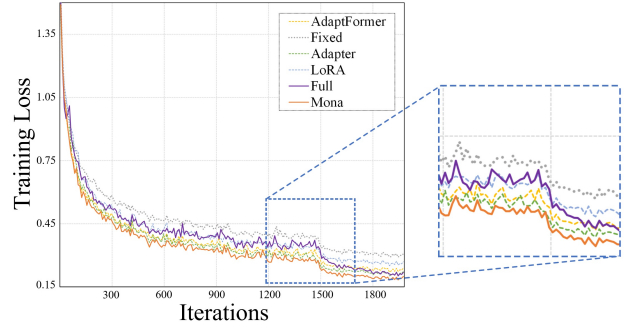


Figure 3. Loss curves. Among all the methods, the proposed method converges faster and significantly exceeds the full fine-tuning.

outperforms all other methods in Table 2. Mona outperforms full fine-tuning by 3.6% and 0.18% on these two tasks. Table 2 again indicates that full fine-tuning is not the best choice for visual transfer learning. Interestingly, all baselines surpasses full fine-tuning on VOC, which is different from COCO and ADE20K. Relatively little data in VOC may lead to over-fitting when full fine-tuning a 198M Swin-Large pre-trained model. Compared to full fine-tuning, other methods fix most pre-trained parameters, so the model performance is less likely to collapse severely during tuning. NLP scholars treat similar cases as low-resource cases [67, 68]. VOC here can be considered as a low-resource case in CV. For ADE20K, the performance gaps between baselines without additional structure and adapter-based baselines are more significant than VOC and COCO. For parameter sizes, most methods in Tables 1 and 2 (except Partial-1) produce less than 5% new backbone parameters, which is the characteristic of delta-

tuning. Despite the slight increase in parameters, Mona still outperforms the previous art and breaks the full fine-tuning performance ceiling by a wide margin.

Table 4 shows the performance on the more challenging oriented object detection tasks. Firstly, Columns 2-3 of Table 4 show that Mona outperforms full fine-tuning and other efficient fine-tuning methods on two datasets with Oriented R-CNN [69]. Secondly, STAR has more instances and classes than DOTA, which is more challenging than DOTA. So we experiment with STAR on more frameworks. Columns 4-5 show the results of all methods with KLD [70] and H2RBox-v2 [71]. It can be seen that Mona outperforms all baseline methods on these frameworks. Table 4 further illustrates that the proposed adapter can lead to performance breakthroughs on a wider range of visual tasks.

For classification tasks, we show the individual and average results on three classification datasets in Table 3. Mona outperforms all the baselines on Flowers102, OxfordPets, and outperforms the average results of all baselines. Table 3 indicates that Mona has a high transfer efficiency on relatively simple tasks. In addition, we find that the average results of all delta-tuning methods surpass full fine-tuning, which is similar to conclusions in previous art [18]. Compared to classification, the complex dense prediction tasks can demonstrate the advantages and disadvantages of different tuning approaches more intuitively.

In summary, the results of Tables 1 to 4 can be summarized in two aspects: 1) As to performance, the widely used full fine-tuning paradigm in art like Swin is no longer the optimal choice for visual tasks. The proposed Mona-tuning surpasses the performance ceiling of full fine-tuning in representative tasks such as instance/semantic segmentation, object detection, image classification, and oriented object detection. Specifically, Mona achieves a 1% AP gain over full fine-tuning in the challenging COCO instance segmentation task. 2) Mona, based on multi-cognitive visual filtering, surpasses recent remarkable baselines in most tasks. Mona comprehensively enhances the practicality and generality of delta-tuning in visual tasks. Mona-tuning not only significantly reduces storage costs, but also further elevates the performance ceiling of visual tasks.

4.5. Loss Analysis

We present the loss converging process for Mona and five representative baselines on the object detection task (Pascal VOC) in Figure 3. The proposed method yields a significant advantage in the convergence process compared to full fine-tuning, which explains its better performance on VOC. Mona also converges faster than other delta-tuning methods, suggesting that multi-cognitive visual filters can better process visual features and accelerate the convergence of transfer learning. Convergence analysis again demonstrates that the proposed method is a highly competitive

Intermediate Dimensions	Trained Params*	AP _{Box}
32	1.35 %	86.8 %
64	2.56 %	87.3 %
128	5.22 %	87.1 %

Table 5. **Ablations of intermediate dimensions.** 64 intermediate dimensions achieves the best performance. * denotes the trainable parameters in backbones.

Model	FULL (VOC)	MONA (VOC)	Param % (Mona)
Swin-T	80.1 %	83.5 %	4.87 %
Swin-B	81.6 %	86.5 %	4.06 %
Swin-L	83.7 %	87.3 %	2.56 %

Table 6. **Performance of mona on models with different sizes.** The results indicate that model sizes do not constrain Mona’s superiority.

visual transfer learning method and full fine-tuning is no longer the optimal choice for visual tasks.

4.6. Ablations

In this section, we ablate some potential factors that affect model performance, including intermediate dimensions, model sizes, detailed designs and frameworks. All ablation experiments are conducted on Pascal VOC.

Intermediate Dimension. The workflow of the adapter is to compress the input from pre-trained layers into a low-dimensional feature space and transfer the pre-trained knowledge by tuning adapters. Thus, the intermediate dimension is important for adapters. We ablate the intermediate dimension of Mona in Table 5 and fix other settings. Dimension candidates are 32, 64, and 128. Table 5 show that the 64-dimension surpasses 32-dimension and 128-dimension. Chen et al. [19] also study the intermediate dimension of AdaptFormer. They find that the 64-dimension AdaptFormer surpasses its 32- and 256-dimension versions in classification tasks, which is consistent with our results. The results of Table 5 and Chen et al. indicate that the intermediate dimension of the adapter is not proportional to the performance, which means that a larger number of adapter parameters does not necessarily lead to better results.

Model Sizes. In Table 6, we change the size of the backbones under the same settings, and the model candidates are 29M Swin-T, 88M Swin-B, and 197M Swin-L. We can draw the following three conclusions from Table 6. First, the more parameters the backbone network has, the smaller the proportion of Mona parameters for the same Mona setting is. This result indicates that Mona-tuning can save more parameters when the backbone gets larger. Existing visual models are getting larger and larger. InternImage-H

Scaled Norm	Kernel Settings	AP _{Box}
×	[3,5,7]	86.9 %
✓	[3,5,7]	87.3 %
✓	[3]	86.9 %
✓	[5]	87.0 %
✓	[7]	86.7 %
✓	[3,3,3]	86.8 %
✓	[3,5]	86.9 %
✓	[5,7]	87.1 %

Table 7. **Ablations of detailed structures in Mona.** Scaled normalization effectively improves performance. [3,5,7] performs best in all kernel settings.

[72] reaches 1.08B parameters, and SwinV2-G [73] reaches 3B. Parameter-efficient Mona-tuning can save billions of parameters and massive storage costs in the era of large models. Second, Mona surpasses full fine-tuning under all settings, and its performance improves when model size grows. Table 6 shows that Mona-tuning can improve training efficiency and performance in smaller models. We just discussed Mona’s advantages for large models. However, many resource-limited research teams and project groups use small models. Mona-tuning can also help resource-limited researchers leverage high-performance large models in their own applications. Third, the proposed method is more capable of stimulating the potential of large models compared to full fine-tuning. From Swin-T to Swin-L, full fine-tuning brings 3.6% performance gain, while Mona brings 3.8%. In other words, Mona can perform better as the model gets larger and help further increase the upper bound for performance-sensitive tasks.

Detailed Designs of Mona. Table 7 presents ablation experiments on two detailed designs of Mona, including the scaled normalization and the convolution kernel setting. The first two rows demonstrate that scaled normalization effectively enhances Mona’s performance. The subsequent seven rows compare Mona’s performance under different convolution kernel settings of the multi-cognitive visual filters (each setting takes the average of its convolution results). The fifth row indicates that using excessive convolution kernels does not necessarily result in performance gains. Among the seven settings, [3,5,7] performs best and is adopted in Mona. The ablation of details demonstrate the rationality and necessity of the Mona structure.

Results on PVT. We also test Mona on another influential visual framework, PVT [74], to demonstrate the generalizability of the proposed approach. Table 8 shows that Mona outperforms not only full fine-tuning but also other baselines on PVT (PVT-Large), which indicates that Mona is effective on a wider range of visual frameworks.

Methods	AP _{Box}
Full (PVT-L)	76.1%
Adapter (PVT-L)	78.9%
LoRA (PVT-L)	77.6%
AdaptFormer (PVT-L)	79.2%
LoRand (PVT-L)	79.3%
Mona (PVT-L)	80.3%

Table 8. **Results on PVT.** Mona outperforms other competitive baseline methods on PVT, demonstrating that Mona can achieve similar results across a broader range of visual frameworks.

5. Discussion

Inference cost is a concern for the industry. Methods based on adapter modules (such as Adapter, AdaptFormer, LoRand, and Mona) introduce additional structures, along with a small increase in inference cost. Reparameterization-based approaches (like LoRA) do not affect inference cost but tend to perform poorly on visual tasks. The utility of adapters could be greatly enhanced by combining the strengths of both approaches. We will continue to strive towards achieving this goal.

Additionally, we are also interested in discussing the value of delta-tuning in mobile applications. In the era of large models, storing a single Swin-H [73] requires 2.5GB, and Swin-G even reaches 4GB. However, the Top-10 U.S. apps required only collectively 2.2GB in May 2021 [47]. Recently, many manufacturers have incorporated AI chips into mobile devices to enable large model inference. With delta-tuning, mobile devices only need to store one large model and several small structures (such as Mona) in their hard drives or memory, while achieving impressive performance across multiple complex tasks like style migration and portrait beautification. We hope that visual efficient fine-tuning will not only break the performance ceiling of full fine-tuning but also accelerate the deployment and popularization of large visual models on mobile devices.

6. Conclusion

This paper proposes a novel visual fine-tuning method, the multi-cognitive visual adapter (Mona) tuning, which effectively enhances the efficiency and performance of visual fine-tuning. Comprehensive experiments demonstrate that the proposed Mona outperforms traditional full fine-tuning paradigms and other delta-tuning methods across representative tasks, including instance segmentation, semantic segmentation, object detection, image classification, and oriented object detection. In the era of large models, full fine-tuning is no longer the optimal choice for visual tasks. We hope that Mona-tuning can improve the knowledge transferring efficiency of large models and bring performance breakthroughs on more visual tasks.

References

- [1] Jindong Wang and Yiqiang Chen. Pre-training and fine-tuning. In *Introduction to Transfer Learning: Algorithms and Practice*, 2022. 1
- [2] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*, 2021. 1, 2, 4
- [3] Yuxin Fang, Wen Wang, Binhui Xie, Quan Sun, Ledell Wu, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. Eva: Exploring the limits of masked visual representation learning at scale. In *CVPR*, 2023. 1
- [4] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. Using pre-trained models to boost code review automation. In *ICSE*, 2022. 1
- [5] Bonan Min, Hayley Ross, Elinor Sulem, Amir Poursan Ben Veyseh, Thien Huu Nguyen, Oscar Sainz, Eneko Agirre, Ilana Heintz, and Dan Roth. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 2023.
- [6] Robert Tinn, Hao Cheng, Yu Gu, Naoto Usuyama, Xiaodong Liu, Tristan Naumann, Jianfeng Gao, and Hoifung Poon. Fine-tuning large neural language models for biomedical natural language processing. *Patterns*, 2023. 1
- [7] Dongshuo Yin, Xueting Han, Bin Li, Hao Feng, and Jing Bai. Parameter-efficient is not sufficient: Exploring parameter, memory, and time efficient adapter tuning for dense predictions. *ACM MM*, 2023. 1
- [8] Chompunuch Sarasaen, Soumick Chatterjee, Mario Breittkopf, Georg Rose, Andreas Nürnberger, and Oliver Speck. Fine-tuning deep learning model parameters for improved super-resolution of dynamic mri with prior-knowledge. *Artificial Intelligence in Medicine*, 2021. 1
- [9] Caisse Amisse, Mario Ernesto Jijón-Palma, and Jorge Antonio Silva Centeno. Fine-tuning deep learning models for pedestrian detection. *Boletim de Ciências Geodésicas*, 2021.
- [10] Edna Chebet Too, Li Yujian, Sam Njuki, and Liu Yingchun. A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture*, 2019.
- [11] Christoph Käding, Erik Rodner, Alexander Freytag, and Joachim Denzler. Fine-tuning deep neural networks in continuous learning scenarios. In *ACCV*, 2016. 1
- [12] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 2023. 2
- [13] Shengding Hu, Zhen Zhang, Ning Ding, Yadao Wang, Yasheng Wang, Zhiyuan Liu, and Maosong Sun. Sparse structure search for delta tuning. *NeurIPS*, 2022. 2
- [14] Xin Zhou, Ruotian Ma, Yicheng Zou, Xuanning Chen, Tao Gui, Qi Zhang, Xuan-Jing Huang, Rui Xie, and Wei Wu. Making parameter-efficient tuning more efficient: A unified framework for classification tasks. In *International Conference on Computational Linguistics*, 2022. 2
- [15] Himashi Rathnayake, Janani Sumanapala, Raveesha Rukshani, and Surangika Ranathunga. Adapter-based fine-tuning of pre-trained multilingual language models for code-mixed and code-switched text classification. *Knowledge and Information Systems*, 2022. 2
- [16] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *ECCV*, 2022. 2, 3
- [17] Yen-Cheng Liu, Chih-Yao Ma, Junjiao Tian, Zijian He, and Zsolt Kira. Polyhistor: Parameter-efficient multi-task adaptation for dense vision tasks. *NeurIPS*, 2022. 2
- [18] Xuehai He, Chunyuan Li, Pengchuan Zhang, Jianwei Yang, and Xin Eric Wang. Parameter-efficient fine-tuning for vision transformers. *arXiv preprint arXiv:2203.16329*, 2022. 2, 7
- [19] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. Adaptformer: Adapting vision transformers for scalable visual recognition. *NeurIPS*, 2022. 2, 3, 5, 7
- [20] Dongshuo Yin, Yiran Yang, Zhechao Wang, Hongfeng Yu, Kaiwen Wei, and Xian Sun. 1% vs 100%: Parameter-efficient low rank adapter for dense predictions. In *CVPR*, 2023. 2, 3, 5
- [21] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *ICML*, 2019. 2, 3, 5
- [22] Jiuxiang Gu, Zhenhua Wang, Jason Kuen, Lianyang Ma, Amir Shahroudy, Bing Shuai, Ting Liu, Xingxing Wang, Gang Wang, Jianfei Cai, et al. Recent advances in convolutional neural networks. *Pattern recognition*, 2018. 2
- [23] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE TNNLS*, 2021.
- [24] Saad Albawi, Tareq Abed Mohammed, and Saad Al-Zawi. Understanding of a convolutional neural network. In *ICET*, 2017. 2
- [25] Şaban Öztürk, Umut Özkaya, Bayram Akdemir, and Levent Seyfi. Convolution kernel size effect on convolutional neural network in histopathological image processing applications. In *ISFEE*, 2018. 2
- [26] Danupon Chansong and Siriporn Supratid. Impacts of kernel size on different resized images in object recognition based on convolutional neural network. In *IEEECON*, 2021.
- [27] Abhinav Agrawal and Namita Mittal. Using cnn for facial expression recognition: a study of the effects of kernel size and number of filters on accuracy. *The Visual Computer*, 2020. 2
- [28] Xue Yang, Jirui Yang, Junchi Yan, Yue Zhang, Tengfei Zhang, Zhi Guo, Xian Sun, and Kun Fu. Srdet: Towards more robust detection for small, cluttered and rotated objects. In *ICCV*, 2019. 2
- [29] Xue Yang, Junchi Yan, Ziming Feng, and Tao He. R3det: Refined single-stage detector with feature refinement for rotating object. In *AAAI*, 2021. 2
- [30] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, 2009. 2, 4

- [31] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *ECCV*, 2014. 2, 4
- [32] Luciano Floridi and Massimo Chiriatti. Gpt-3: Its nature, scope, limits, and consequences. *Minds and Machines*, 2020. 2
- [33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [34] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. Segment anything. *arXiv preprint arXiv:2304.02643*, 2023. 2
- [35] Tianrun Chen, Lanyun Zhu, Chaotao Deng, Runlong Cao, Yan Wang, Shangzhan Zhang, Zejian Li, Lingyun Sun, Ying Zang, and Papa Mao. Sam-adapter: Adapting segment anything in underperformed scenes. In *ICCV*, 2023. 2
- [36] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021. 2, 5
- [37] Chongjie Si, Xuehui Wang, Xue Yang, Zhengqin Xu, Qingyun Li, Jifeng Dai, Yu Qiao, Xiaokang Yang, and Wei Shen. Flora: Low-rank core space for n-dimension. *arXiv preprint arXiv:2405.14739*, 2024. 2
- [38] Yi Xin, Siqi Luo, Haodi Zhou, Junlong Du, Xiaohong Liu, Yue Fan, Qing Li, and Yuntao Du. Parameter-efficient fine-tuning for pre-trained vision models: A survey. *arXiv preprint arXiv:2402.02242*, 2024. 2
- [39] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. *arXiv preprint arXiv:2106.10199*, 2021. 2, 4
- [40] Angeliki Giannou, Shashank Rajput, and Dimitris Papailiopoulos. The expressive power of tuning only the norm layers. *arXiv preprint arXiv:2302.07937*, 2023. 2, 4
- [41] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? *NeurIPS*, 2014. 2, 4
- [42] Xiao Liu, Kaixuan Ji, Yicheng Fu, Weng Tam, Zhengxiao Du, Zhilin Yang, and Jie Tang. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *ACL*, 2022. 2
- [43] Beier Zhu, Yulei Niu, Yucheng Han, Yue Wu, and Hanwang Zhang. Prompt-aligned gradient for prompt tuning. In *ICCV*, 2023. 2
- [44] Yi-Lin Sung, Jaemin Cho, and Mohit Bansal. Vl-adapter: Parameter-efficient transfer learning for vision-and-language tasks. In *CVPR*, 2022. 2
- [45] Xuehai He, Chunyuan Li, Pengchuan Zhang, Jianwei Yang, and Xin Eric Wang. Parameter-efficient model adaptation for vision transformers. In *AAAI*, 2023. 2
- [46] Zhe Chen, Yuchen Duan, Wenhai Wang, Junjun He, Tong Lu, Jifeng Dai, and Yu Qiao. Vision transformer adapter for dense predictions. In *ICLR*, 2022. 2
- [47] Haoyu He, Jianfei Cai, Jing Zhang, Dacheng Tao, and Bohan Zhuang. Sensitivity-aware visual parameter-efficient fine-tuning. In *ICCV*, 2023. 2, 8
- [48] Zhi Zhang, Qizhe Zhang, Zijun Gao, Renrui Zhang, Ekaterina Shutova, Shiji Zhou, and Shanghang Zhang. Gradient-based parameter selection for efficient fine-tuning. In *CVPR*, 2024. 2
- [49] Dongze Lian, Daquan Zhou, Jiashi Feng, and Xinchao Wang. Scaling & shifting your features: A new baseline for efficient model tuning. *NeurIPS*, 2022. 2
- [50] Fengze Jiang, Shuling Wang, and Xiaojin Gong. Task-conditional adapter for multi-task dense prediction. In *ACM MM*, 2024. 2
- [51] Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin. Understanding and improving layer normalization. *NeurIPS*, 2019. 3
- [52] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015. 3
- [53] Jane F Koretz and George H Handelman. How the human eye focuses. *Scientific American*, 1988. 3
- [54] Susana Martinez-Conde, Stephen L Macknik, and David H Hubel. The role of fixational eye movements in visual perception. *Nature reviews neuroscience*, 2004.
- [55] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *CVPR*, 2015. 3
- [56] Mark Everingham, SM Eslami, Luc Van Gool, Christopher KI Williams, John Winn, and Andrew Zisserman. The pascal visual object classes challenge: A retrospective. *IJCV*, 2015. 4
- [57] Bolei Zhou, Hang Zhao, Xavier Puig, Sanja Fidler, Adela Barriuso, and Antonio Torralba. Scene parsing through ade20k dataset. In *CVPR*, 2017. 4
- [58] Jian Ding, Nan Xue, Gui-Song Xia, Xiang Bai, Wen Yang, Michael Yang, Serge Belongie, Jiebo Luo, Mihai Datcu, Marcello Pelillo, and Liangpei Zhang. Object detection in aerial images: A large-scale benchmark and challenges. *IEEE TPAMI*, 2021. 4
- [59] Yansheng Li, Linlin Wang, Tingzhu Wang, Xue Yang, Junwei Luo, Qi Wang, Youming Deng, Wenbin Wang, Xian Sun, Haifeng Li, Bo Dang, Yongjun Zhang, Yu Yi, and Junchi Yan. Star: A first-ever dataset and a large-scale benchmark for scene graph generation in large-size satellite imagery. *arXiv preprint arXiv: 2406.09410*, 2024. 4
- [60] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian Conference on Computer Vision, Graphics & Image Processing*, 2008. 4
- [61] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In *CVPR*, 2012. 4
- [62] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman. The PASCAL Visual Object Classes Challenge 2007 (VOC2007) Results. <http://www.pascal-network.org/challenges/VOC/voc2007/workshop/index.html>, 2007. 4

- [63] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, et al. Mmdetection: Open mmlab detection toolbox and benchmark. *arXiv preprint arXiv:1906.07155*, 2019. 4
- [64] MMSegmentation Contributors. MMSegmentation: Openmmlab semantic segmentation toolbox and benchmark. <https://github.com/open-mmlab/mmdetection>, 2020. 4
- [65] Yue Zhou, Xue Yang, Gefan Zhang, Jiabao Wang, Yanyi Liu, Liping Hou, Xue Jiang, Xingzhao Liu, Junchi Yan, Chengqi Lyu, Wenwei Zhang, and Kai Chen. Mmrotate: A rotated object detection benchmark using pytorch. In *ACM MM*, 2022. 4
- [66] MMClassification Contributors. Openmmlab's image classification toolbox and benchmark. <https://github.com/open-mmlab/mmdetection>, 2020. 4
- [67] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. Fine-tuning pre-trained language models: Weight initializations, data orders, and early stopping. *arXiv preprint arXiv:2002.06305*, 2020. 6
- [68] Matthew E Peters, Sebastian Ruder, and Noah A Smith. To tune or not to tune? adapting pretrained representations to diverse tasks. *arXiv preprint arXiv:1903.05987*, 2019. 6
- [69] Xingxing Xie, Gong Cheng, Jiabao Wang, Xiwen Yao, and Junwei Han. Oriented r-cnn for object detection. In *ICCV*, 2021. 7
- [70] Xue Yang, Xiaojiang Yang, Jirui Yang, Qi Ming, Wentao Wang, Qi Tian, and Junchi Yan. Learning high-precision bounding box for rotated object detection via kullback-leibler divergence. *NeurIPS*, 2021. 7
- [71] Yi Yu, Xue Yang, Qingyun Li, Yue Zhou, Feipeng Da, and Junchi Yan. H2rbox-v2: Incorporating symmetry for boosting horizontal box supervised oriented object detection. *NeurIPS*, 2024. 7
- [72] Wenhai Wang, Jifeng Dai, Zhe Chen, Zhenhang Huang, Zhiqi Li, Xizhou Zhu, Xiaowei Hu, Tong Lu, Lewei Lu, Hongsheng Li, et al. Internimage: Exploring large-scale vision foundation models with deformable convolutions. In *CVPR*, 2023. 8
- [73] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, et al. Swin transformer v2: Scaling up capacity and resolution. In *CVPR*, 2022. 8
- [74] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *ICCV*, 2021. 8